OBSERVATION – 7

Demonstrate 10 Exception Classes in Java with Code Examples

1. Aim

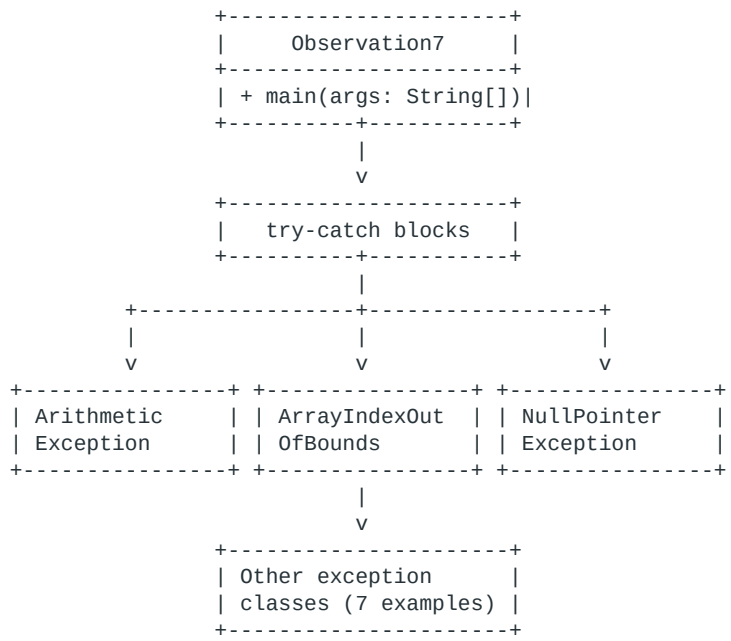
To write and execute a Java program that demonstrates ten different exception classes and shows how exceptions can be handled using **try-catch** blocks.

2. Description

An exception is an event that interrupts the normal flow of a program during execution. Java provides exception classes to represent different kinds of errors. Statements that may cause an exception are placed inside a **try** block, and the matching **catch** block handles it. This program demonstrates ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerException, NumberFormatException, StringIndexOutOfBoundsException, NegativeArraySizeException, ClassCastException, IllegalArgumentException, InputMismatchException, and UnsupportedOperationException.

Note: The examples intentionally cause exceptions for learning purposes and catch them so the program continues running.

3. UML Diagram



UML summary: Observation7 contains the main() method. It executes ten try-catch examples, each designed to trigger and handle one exception class.

4. Source Code with Comments

Save the following program as **Observation7.java**. The public class name and file name must match.

```
// Observation 7: Demonstrate 10 Exception Classes in Java
// Save as Observation7.java

public class Observation7 {
    public static void main(String[] args) {
        System.out.println("OBSERVATION 7: JAVA EXCEPTION CLASSES");
        System.out.println("-------------------------------------");

        // 1. ArithmeticException: integer division by zero
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("1. ArithmeticException caught");
        }

        // 2. ArrayIndexOutOfBoundsException: invalid array index
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("2. ArrayIndexOutOfBoundsException caught");
        }

        // 3. NullPointerException: calling a method on null
        try {
            String text = null;
            System.out.println(text.length());
        } catch (NullPointerException e) {
            System.out.println("3. NullPointerException caught");
        }

        // 4. NumberFormatException: invalid numeric conversion
        try {
            int value = Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("4. NumberFormatException caught");
```

4. Source Code with Comments (continued)

```
        }

        // 5. StringIndexOutOfBoundsException: invalid string index
        try {
            String word = "Java";
            System.out.println(word.charAt(8));
        } catch (StringIndexOutOfBoundsException e) {
            System.out.println("5. StringIndexOutOfBoundsException caught");
        }

        // 6. NegativeArraySizeException: negative array size
        try {
            int[] values = new int[-3];
        } catch (NegativeArraySizeException e) {
            System.out.println("6. NegativeArraySizeException caught");
        }

        // 7. ClassCastException: incompatible type cast
        try {
            Object item = Integer.valueOf(25);
            String converted = (String) item;
        } catch (ClassCastException e) {
            System.out.println("7. ClassCastException caught");
        }

        // 8. IllegalArgumentException: invalid method argument
        try {
            Thread.sleep(-1);
        } catch (IllegalArgumentException e) {
            System.out.println("8. IllegalArgumentException caught");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Thread was interrupted");
        }

        // 9. InputMismatchException: input does not match expected type
        try {
            java.util.Scanner scanner =
                new java.util.Scanner("hello");
            int input = scanner.nextInt();
            scanner.close();
        } catch (java.util.InputMismatchException e) {
            System.out.println("9. InputMismatchException caught");
        }

        // 10. UnsupportedOperationException: operation is not supported
        try {
            throw new UnsupportedOperationException("Demo exception");
        } catch (UnsupportedOperationException e) {
            System.out.println("10. UnsupportedOperationException caught");
        }

        System.out.println("-----------------------------------");
        System.out.println("All 10 exception demonstrations completed.");
    }
}
```

5. Terminal-Style Execution

Compile and run the program from the folder containing Observation7.java:

```
$ javac Observation7.java
$ java Observation7
```

6. Output Screenshot (Terminal Style)

● **Terminal — Java Output**

```
OBSERVATION 7: JAVA EXCEPTION CLASSES
-------------------------------------
1. ArithmeticException caught
2. ArrayIndexOutOfBoundsException caught
3. NullPointerException caught
4. NumberFormatException caught
5. StringIndexOutOfBoundsException caught
6. NegativeArraySizeException caught
7. ClassCastException caught
8. IllegalArgumentException caught
9. InputMismatchException caught
10. UnsupportedOperationException caught
-------------------------------------
All 10 exception demonstrations completed.
```

The output above is a terminal-style representation of the expected output after successful compilation and execution.

7. Result

Thus, the Java program was designed to demonstrate ten exception classes and handle each exception using try-catch blocks. The program continues execution after handling each exception.